Redes Neuronales y Aprendizaje Profundo
Curso de Deep Learning
Universidad EAFIT

Laboratorio # 2 – MÓdulo II (Valor: 15%)

Parte A -Transfer Learning con PyTorch

Objetivos

- Entender el concepto de transfer learning y cuándo usarlo.
- Cargar y usar modelos preentrenados con *torchvision.models* usando la API moderna.
- Extraer *features* con una red convolucional fija y entrenar un clasificador sobre ellas.
- Reemplazar el clasificador final y hacer *fine-tuning* por etapas.
- Comparar el comportamiento de VGG-16 y ResNet-18 adaptados a un problema binario
- Visualizar y comparar experimentos con *TensorBoard*.

Descripción de los modelos a desarrollar para el laboratorio

	Modelo	Descripción
M1	VGG-16	VGG-16 como extractor de features fijo + clasificador lineal.
M2	VGG-16	VGG-16 con clasificador reemplazado y fine-tuning completo.
M3	ResNet-18	ResNet-18 adaptado a Cats vs. Dogs, propuesto por el estudiante

Notebook TransferLearning_Workshop.ipynb

Actividad 1 — Dataset y pipeline de preprocesamiento

Usar el dataset Kaggle Cats vs. Dogs. Una vez descargado y extraído, debes verificar y eliminar imágenes corruptas con check_image_dir. Luego construye el pipeline de transforms para preprocesar las imágenes correctamente.

Los modelos preentrenados en ImageNet esperan imágenes de 224×224 normalizadas con media [0.485, 0.456, 0.406] y desviación estándar [0.229, 0.224, 0.225]. El pipeline debe seguir el orden: Resize a 256 → CenterCrop a 224 → ToTensor → Normalize.

Pregunta: ¿de dónde vienen esos valores de media y std? ¿Por qué es importante usarlos si vas a cargar pesos preentrenados?

Actividad 2 — Modelos preentrenados y extracción de features

Carga VGG-16 con pesos preentrenados en ImageNet usando la API moderna de torchvision (la línea con pretrained=True está deprecada desde la versión 0.13, corrígela). Verifica que la salida del modelo para una imagen de muestra tenga shape [1, 1000], luego usa el mapeo de clases de ImageNet para mostrar el nombre de la clase predicha.

Luego usa vgg.features como extractor fijo para pre-computar los vectores de features de 800 imágenes del dataset. Almacénalos en un tensor de shape [800, 512×7×7] usando torch.no_grad() para no calcular gradientes durante la extracción.

¿Cuántos parámetros tiene VGG-16 en total? ¿Qué hace la capa Dropout y por qué mejora la generalización? ¿Por qué la última capa tiene 1000 salidas?

Actividad 3 — Clasificador lineal sobre features precomputadas

Con los features fijos, entrena solo una capa lineal que mapee de 512×7×7 entradas a 2 clases. Este es M1 en su primera etapa.

Ten en cuenta que usar nn.LogSoftmax() al final de la red junto con CrossEntropyLoss es un error: CrossEntropyLoss en PyTorch ya aplica softmax internamente, combinándolos se doble-aplica la operación y produce resultados incorrectos. Define net como una sola capa nn.Linear sin ninguna activación al final. Entrena por 5 épocas y guarda la curva con TensorBoard.

¿Cuál es la diferencia entre CrossEntropyLoss y NLLLoss + LogSoftmax? ¿Por qué combinarlos es un error?

Actividad 4 — Transfer Learning: reemplazar el clasificador y congelar features

Recarga VGG-16 limpio y reemplaza su clasificador final (vgg.classifier) por una capa nn.Linear de 25088 entradas a 2 salidas. Luego congela todos los parámetros de vgg.features poniendo requires_grad = False en cada uno, de modo que solo el nuevo clasificador se entrañe.

Verifica con summary() que el número de parámetros entrenables sea aproximadamente 50k. Entrena por 1 época sobre el dataset completo y guarda el modelo.

Nota: el summary() del código base usa tamaño 244 en lugar de 224. Corrígelo antes de ejecutarlo.

¿Por qué es importante congelar las capas convolucionales antes de entrenar el nuevo clasificador?

Actividad 5 — Fine-tuning completo

Carga el modelo guardado en la actividad anterior y descongela los parámetros de vgg.features poniendo requires_grad = True. Entrena por 1 época adicional con un learning rate más pequeño (1e-4) para no destruir los pesos preentrenados.

Compara las curvas de esta etapa con las de la etapa congelada en TensorBoard.

Actividad 6 — Adaptar ResNet-18 (M3)

ResNet-18 está disponible en torchvision.models. A diferencia de VGG, su clasificador final se llama fc en lugar de classifier. Congela todos los parámetros de la red, investiga cuántas entradas tiene resnet18.fc, y reemplázalo con una capa lineal hacia 2 clases. Entrena solo el clasificador por 2 épocas y compara con VGG en velocidad y accuracy.

¿Qué es un residual block? ¿Qué problema resuelve en redes muy profundas? ¿ResNet-18 entrena más rápido que VGG? ¿Por qué?

Preguntas de investigación

1. ¿Cuándo NO conviene usar transfer learning? Da al menos dos escenarios concretos.
2. ¿Qué transforms de data augmentation añadirías al pipeline de entrenamiento para mejorar generalización? Impleméntalos y experimenta.
3. Investiga StepLR y CosineAnnealingLR. Agrega uno al loop de fine-tuning de la Actividad 5 y compara curvas en TensorBoard.
4. ¿Por qué ResNet usa BatchNorm y VGG no? ¿Qué efecto tiene en el entrenamiento?
5. Reto opcional: prueba ResNet-50 o EfficientNet-B0 y compara accuracy final con ResNet-18.

Entregables

Archivo — Notebook (grupoX_LAB#2A.ipynb)

Un único notebook .ipynb con todo el código completo y ejecutado, que incluya las respuestas a todas las preguntas en **celdas Markdown.**

El reporte debe ser autocontenido: alguien que no ejecute el notebook debe poder entender la solución planteada.

Parte B - Segmentación Semántica con U-Net y Transfer Learning

Objetivos

- Entender la diferencia entre clasificación de imágenes y segmentación semántica (etiqueta por píxel).
- Construir los bloques fundamentales de U-Net en PyTorch: DoubleConv, UpBlock y skip connections.
- Comprender por qué las funciones de pérdida estándar fallan ante desbalance de clases y cómo resolverlo.
- Conectar lo aprendido en el workshop anterior: reutilizar un encoder ResNet-18 preentrenado dentro de una U-Net.
- Evaluar modelos de segmentación con métricas apropiadas: IoU y Dice score.

Descripción de los modelos a desarrollar para el laboratorio

	Modelo	Descripción
M1	U-Net	U-Net desde cero (encoder aleatorio, línea base).
M2	ResNetUNet	ResNetUNet con encoder congelado (solo se entrena el decoder).
M3	ResNetUNet	ResNetUNet con fine-tuning completo (encoder + decoder).

Notebook UNet_Workshop.ipynb

Actividad 1 — Dataset y transforms sincronizados

Usar el Oxford-IIIT Pet Dataset, que incluye imágenes de mascotas con máscaras de segmentación píxel a píxel. Cada píxel tiene una de tres etiquetas: primer plano (el animal), fondo, o borde ambiguo. Lo tratarás como un problema binario: el píxel pertenece al animal o no.

Antes de escribir código, ejecuta show_sample sobre al menos 3 imágenes y observa cómo se ve la máscara cruda y el overlay binario. Entiende qué representa cada valor numérico en la máscara antes de continuar.

El detalle crítico de esta actividad es que imagen y máscara deben recibir exactamente las mismas transformaciones geométricas. Si aplicas un flip o una rotación de forma independiente, la máscara queda desalineada con la imagen. La clase PetSegDataset ya implementa esto: estudia cómo lo hace antes de usarla.

¿Por qué se usa interpolación NEAREST para la máscara y BILINEAR para la imagen al hacer resize? ¿Qué pasaría si usaras bilineal para ambas?

Actividad 2 — Construir U-Net desde cero

La U-Net tiene forma de «U»: un encoder que reduce la resolución espacial extrayendo features, y un decoder que la recupera progresivamente. La clave son las skip connections, que conectan cada nivel del encoder con su nivel correspondiente en el decoder mediante concatenación.

Los bloques DoubleConv y UpBlock ya están implementados. Tu tarea es completar el método forward de la clase UNet. El encoder ya está escrito y guarda las skip connections en una lista llamada skips. Debes iterar sobre self.decoders usando esas skips en orden inverso: la última skip del encoder corresponde al primer bloque del decoder.

Verifica tu implementación con la prueba de shapes antes de continuar. La salida debe tener shape [2, 1, 256, 256] para una entrada [2, 3, 256, 256].

¿Qué información aportan las skip connections que el decoder no podría recuperar por sí solo?

Actividad 3 — Funciones de pérdida para segmentación

Con CrossEntropyLoss estándar existe un problema serio en segmentación: si el 90% de los píxeles son fondo, el modelo puede ignorar completamente el objeto de interés y aun así tener 90% de accuracy. Esto es especialmente crítico en imágenes médicas.

Estudia la implementación de DiceLoss y BCEDiceLoss. Entiende por qué el coeficiente Dice es robusto ante el desbalance de clases y cómo la pérdida combinada BCE + Dice estabiliza el entrenamiento. No modifiques estas clases todavía, pero ten claro qué recibe cada una como entrada y qué devuelve, porque las usarás en todas las actividades siguientes.

¿Por qué CrossEntropyLoss falla ante desbalance de clases? ¿Qué mide exactamente el coeficiente Dice?

Actividad 4 — Entrenar U-Net como línea base

Instancia UNet(in_channels=3, num_classes=1) y entrénala con train_seg por 5 epochs usando la pérdida combinada y Adam con lr=1e-3. Esta es tu línea base: M1 con encoder aleatorio, sin ningún conocimiento previo.

Guarda las curvas de loss, IoU y Dice. No necesitas convergencia completa, solo un punto de referencia numérico para comparar contra los modelos con encoder preentrenado.

Actividad 5 — ResNetUNet: encoder preentrenado como backbone

Aquí está la conexión directa con el workshop anterior. En lugar de aprender el encoder desde cero, reutilizas los pesos de ResNet-18 preentrenado en ImageNet. El encoder ya sabe extraer features útiles; el decoder aprende a usarlas para producir la máscara.

Tu tarea es completar el método forward de ResNetUNet. El encoder ya extrae features en cinco niveles (s0 a s4) con sus respectivas resoluciones y canales. Debes implementar el decoder usando dec4 hasta dec0, pasando cada skip connection en el orden correcto. Verifica shapes antes de entrenar.

Luego congela los parámetros del encoder (enc0, enc1, enc2, enc3, enc4) y verifica con summary cuántos parámetros quedan entrenables. Entrena solo el decoder por 5 epochs con lr=1e-3. Guarda el modelo.

¿En qué se diferencia congelar el encoder aquí respecto a lo que hiciste con VGG en el workshop anterior?

Actividad 6 — Fine-tuning end-to-end

Carga el modelo guardado de la actividad anterior y descongela el encoder. Entrena 5 epochs adicionales con lr=1e-4, un learning rate más pequeño para no arruinar los pesos preentrenados. Compara las curvas de esta fase con las de la fase congelada en TensorBoard.

Actividad 7 — Evaluación comparativa y visualización

Con los tres modelos entrenados, evalúa IoU y Dice sobre el test set completo y reporta los resultados en una tabla. Luego usa visualize_predictions para inspeccionar visualmente las predicciones de cada modelo: observa especialmente qué tan precisos son los bordes y dónde aparecen falsos positivos o falsos negativos.

¿Dónde falla más cada modelo: en el interior de la región o en los bordes? ¿A qué atribuyes esa diferencia?

Preguntas de investigación

1. Entrena usando solo BCEWithLogitsLoss sin Dice. ¿Qué pasa con las métricas? ¿Qué estrategias existen para manejar el desbalance en segmentación médica?
2. Elimina las skip connections de ResNetUNet y entrena. ¿Qué observas visualmente en los bordes de las predicciones?
3. La función compute_metrics usa threshold=0.5 para binarizar. Grafica la curva IoU vs. threshold en el test set. ¿Cuál sería el threshold óptimo?
4. Investiga qué augmentations son estándar en imágenes médicas (elastic deformation, gamma correction). Implementa al menos una en PetSegDataset y mide el impacto.
5. Reto opcional: implementa la métrica Hausdorff distance. ¿Qué mide que IoU y Dice no capturan?

Entregables

Archivo — Notebook (grupoX_LAB#2B.ipynb)

Un único notebook .ipynb con todo el código completo y ejecutado, que incluya las respuestas a todas las preguntas en **celdas Markdown.**

El reporte debe ser autocontenido: alguien que no ejecute el notebook debe poder entender la solución planteada.

Observaciones generales

Cada parte del laboratorio tiene un peso del 50%. Por tanto, la nota final será el promedio de las dos partes.

La Parte A debe entregarse a más tardar el **lunes 23 de marzo de 2026 a las 11:59 p. m.**

La Parte B debe entregarse a más tardar el **viernes 27 de marzo de 2026 a las 11:59 p. m.**

Ambas entregas deben realizarse únicamente por correo electrónico, enviándolas al correo institucional del docente. No se aceptarán entregas por Teams, ni por EAFIT Interactiva.

En el asunto del correo debe indicarse exactamente los siguientes formatos:

- Laboratorio #2 - Grupo # - Parte A
- Laboratorio #2 - Grupo # - Parte B

Los envíos realizados después de las fechas estipuladas no serán tenidos en cuenta para la calificación.